

Sample Chapter: Core JDO ISBN 0-13-140731-7

The chapters of the book titled “Core JDO” are being made available for the purposes of review to allow the community to participate in the review and editing of the chapters of the book.

Copyright (c) 2002 Sun Microsystems, Inc. All rights reserved.

FOR PRIVATE USE ONLY: This material is the property of Sun Microsystems, Inc. and is not to be sold, reproduced in any form by any means, or generally distributed without the prior written authorization of Sun Microsystems Inc and Prentice Hall.

To add your comments

- Simply send your uninhibited comments and feedback to the authors at jdoobook@yahoogroups.com
- Or click and add a note using Acrobat where needed. E-mail the PDF back to the authors at jdoobook@yahoogroups.com
- If you would like to receive the chapters in an alternate format, have any questions, concerns or please contact the authors at jdoobook@yahoogroups.com

"It is the beginning of wisdom to be able to say -I do not know". Chinese proverb

JDO Overview

*J*ava Data Objects is not just another specification that has been put together by the community process- it is a new standard for persistence in Java applications. JDO is well placed to change the fundamental way in which architects and developers have approached the design and development of persistence in Java applications. This chapter will give you an insight into the concepts around which JDO is centered

1.1 JDO BACKGROUND

The wheels for an object persistence specification were put into motion in June 1999 under the Java Community Process (JCP) with both individuals and corporations contributing in the development. The first version of the JDO specification, the toolkit and a reference implementation was released in April 2002.

There are already a dozen or so vendor implementations of the JDO standard available including a few open source releases. A brief summary and comparison of these products can be found in Appendix A. The developer community is growing with more and more developers gravitating to the simplicity and ease of use which JDO has exposed. A good place to explore and exchange ideas is <http://www.jdocentral.com> and the alias jdo-interest@java.sun.com

1.2 DOMAIN OBJECT MODEL

A term used frequently when describing JDO is *domain object model*. A domain model is an abstraction that identifies "real-world" entities and their relationships in the context of the business. Architects use the domain model to capture the subject matter expert or domain expert's perspective of the system. For example a broker or stock trader actually using a trading system will identify the logical structure in a trading system.

A domain model is not intended for developers as a result it is not supposed to include any implementation specific details (for example logging or security context).

This can be confusing because the static structure of a domain model is usually represented using UML class models. However, there would normally never be a source code representation of the domain model. An entity from a domain model will generally have multiple representations in different parts of a system. For example, a domain model for a trading system might show an account and a customer with a relationship between them, but the domain model represents the true real-world concepts of 'account' and 'customer'. In the working system, there may be many manifestations of these real-world entities for example a customer widget object on the client side, a customer EJB or business object in the middle tier and a database record in the resource tier. There may be a variety of other analysis, design, and implementation models that reflect how these domain entities are represented and manipulated by the system, but only one domain model.

Figure 1a shows a simple domain model in UML notation for a stock trading system used by a broker. Figure 1b shows another domain model for a system used by agents to process returned orders in UML notation with members expressed in the Java programming language. In general, the abstractions represented by the domain model are turned into one or more *object models* which can be expressed in a programming language and used by developers.

Based on the domain model, business requirements and their own object models, developers will agree that some of the entities represented in their *application layer* require *persistence*, in the sense that the data represented by these classes needs to be available beyond the life of the physical software process (like the JVM) that they are being used in. For example the order, account and customer information needs to be saved so that it can be accessed at a later time when the user checks status. At the same application layer there could be other programming constructs that require persistence too. For example, log messages, security audit trails, user sessions etc.

This is where JDO comes in. It provides developers with a mechanism for *transparent persistence*.

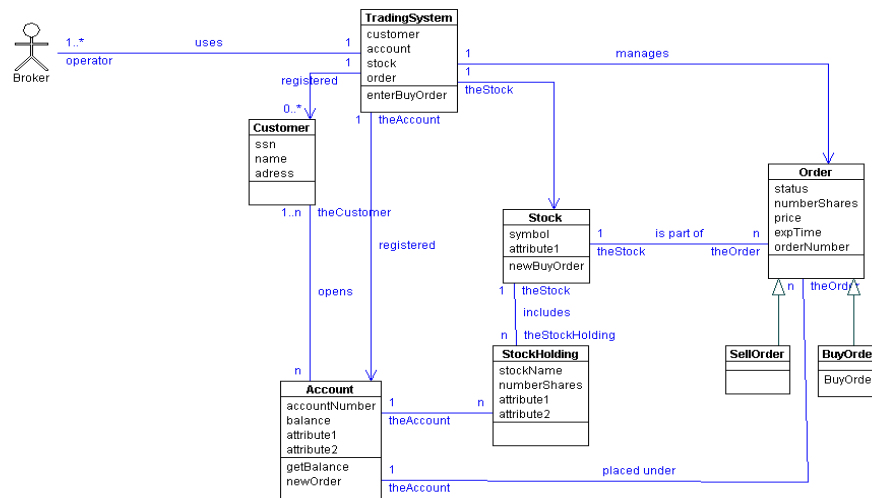


Figure 1(a) A simple stock trading domain model

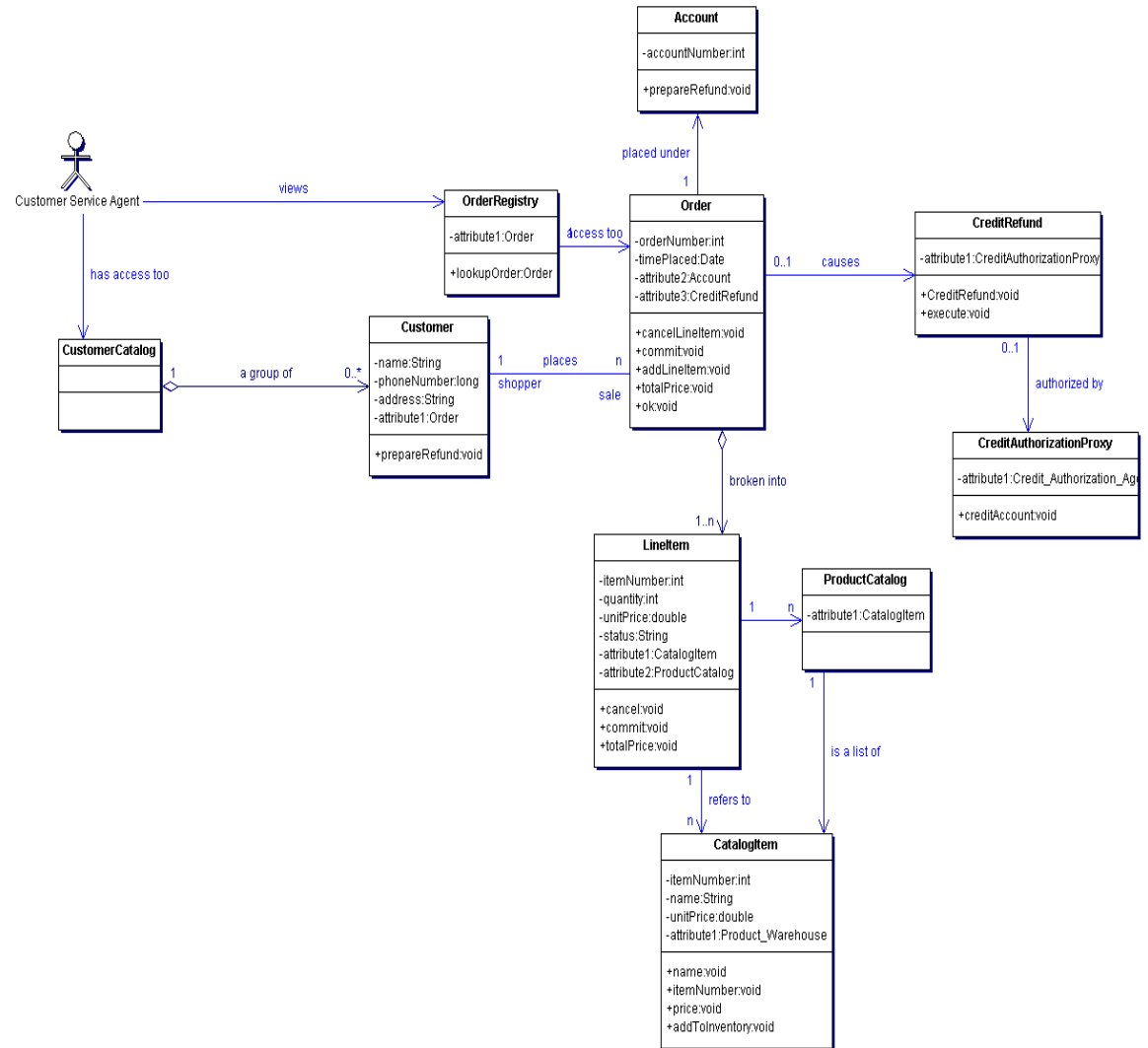


Figure 1(b) An order entry domain model with Java notation

1.3 ORTHOGONAL PERSISTENCE

Java is an object oriented language and developers inherently work with objects, which at any point in time have state represented by member variables. From the Java developers perspective persistence focuses on the fact that the state in these objects should be available once the Java virtual machine has exited. For example an application passing an using an instance of an `Order` class should be capable of recreating the `Order` instance after a machine reboot, or a year later when the account is audited.

The traditional programming approach has been to acquire the relevant state information and store it externally in some form, for example using a relational database. When required, a new object instance is created and populated with this previously stored state. It is the developer's responsibility to map the information in the underlying data store to its programmatic representation.

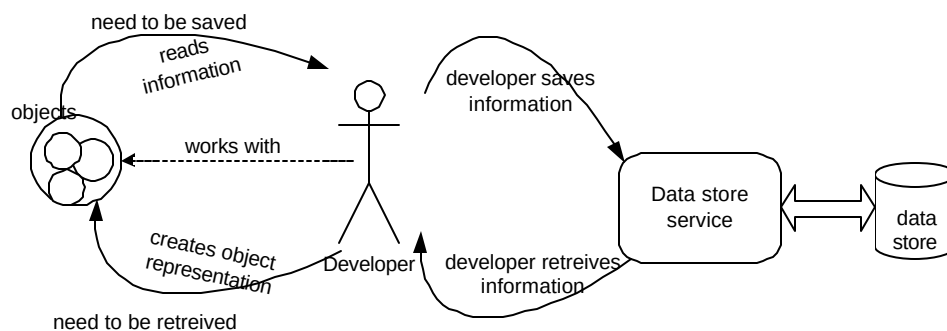


Figure 2: Traditional persistence deals with information

For example, to save an `Order` object from Figure 1b, developers will write code to insert order attributes (`date`, `amount` etc) and dependent attributes like those in the `LineItem` in some data store. When needed the data store will be queried for some fields (like an `orderid`) that uniquely identify the collective information set about that order and a new `Order` instance recreated to be populated with that information.

In general Java applications have followed variations of the following development steps:

- Identify the object model
- Identify persistent entities in the model
- Write database schemas to map the persistent entities information
- Write code to store that information in a form that is understood by the database (e.g. access attributes in objects and insert corresponding rows in a database table) and write code to access that information in a form that can be used by the application (e.g. read rows in a table and return Java objects).
- Write application code that uses these classes and performs the business task.

As shown in Figure 2, developers now have to define, build and use two models that programmatically reflect the same business domain, for example business objects in the application code and their relational representation in the database. They also have to ensure that the view and behavior of these two models remains consistent since the slightest variation in either will result in an impedance mismatch between the models and the real world business.

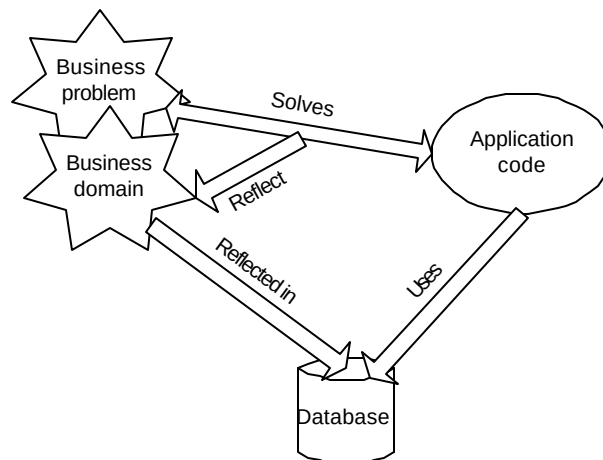


Figure 2 Traditional persistence leads to two models

The idea of *orthogonal persistence* is one in which the application design is decoupled and independent of the entire underlying persistence infrastructure and has been around for a while¹. An application is said to exhibit orthogonal persistence when the following three principles^{2,3} which have been established after many years² of research into persistence are met:

- The principle of data type orthogonality- requires that all data irrespective of its type information should have the ability to be persisted. There are no special cases where objects are not allowed to be *persistent* or are not allowed to be *transient*.
- The principle of persistence independence- requires that for the application code it should be indistinguishable whether it is manipulating persistent data, i.e. data originating from the stable store and outliving the program, or on data that is transient, i.e. data that is created in, and will only exist during the lifetime of the Java program execution.
- The principle of persistence by reachability- requires that the lifetime of objects is determined by their reachability. This is the same principle that is applied to objects on the Java

6

1. Atkinson, M.P., Bailey, P.J., Chisholm, K.J., Cockshott, W.P. & Morrison, R. "An Approach to Persistent Programming". Computer Journal 26, 4 (1983).

2. Atkinson, M. P.; Daynes, L.; Jordan, M. J.; Printezis, T.; Spence, S.: "An orthogonally persistent Java". ACM SIGMOD Record, December 1996. (page 2-3)

3. The Forrest project Sun Laboratories <http://research.sun.com/forest/index.html>

heap, when an object is marked for deletion, the entire tree of objects that it references can be garbage collected.

The first attempts at building orthogonal persistence using these principles in Java were initiated by Sun Laboratories in July 1995 as a research project named “Forest” resulted in a prototype design called PJama³. However the design was based around a modified virtual machine and a second research project called JSPIN⁴ tried another approach with the use of a preprocessor based on earlier research on ADA83⁴. Though JDO is unrelated to these efforts, there is a conceptual similarity in the principles outlined in all these works JDO. In practical terms, JDO fulfills the needs of the community with a much simpler architecture.

An architecture utilizing any form of orthogonal persistence is quite different from the traditional persistence approach we looked at earlier, in that this model deals with objects directly rather than the information contained therein. Such a persistence service will abstract away all the semantics relating to persistence from the application. Rather than working with information and mapping that back and forth from objects, developers work with *persistent objects* directly in their code just like any other application objects. The difference is that some of these objects are now represent persistent information in a data store. The details about how they are persisted and recreated are *transparent* to the developer and this is often referred to as transparent persistence.

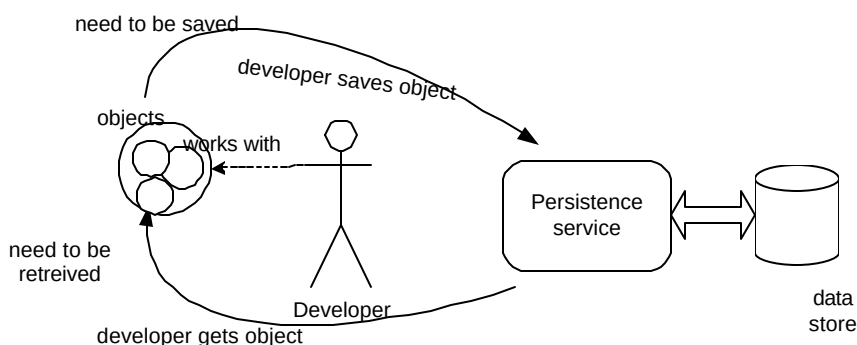


Figure 3: Transparent persistence deals directly with objects

For example, to save an `Order` object from Figure 1b, developers would use a transparent persistence service and give it that particular `Order` instance. When needed by the application at a later time developers would query the service for that unique `Order`, for example using an attribute -“give me the order object with the `orderId` of 2828” and the previously stored object matching that criteria is returned by the service.

JDO is the standard which provides transparent persistence services to Java applications.

With JDO, architects and developers can concentrate on the application at hand, rather than the information contained therein. In comparison to the previously listed steps, development with JDO would involve:

- Identify the object model
- Identify persistent entities in the model
- Write application code that uses these objects and performs the business task
- Use the rather straightforward JDO API in application code to persist and retrieve the objects when needed.

As shown in Figure 4 design with transparent persistence is much more simplified. Developers only need to write and maintain a single model or programmatic representation e.g. business objects in the application code, when needed entities from that model can be persisted using the transparent persistence service.

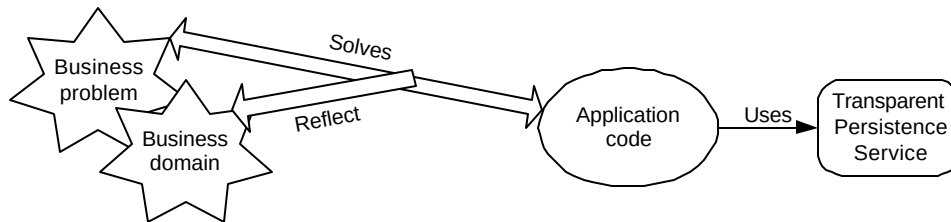


Figure 4: Transparent persistence model

Of course JDO is not the single solution to all persistence related issues and some situations where this might not be appropriate are discussed in Chapter X. However one thing is clear, JDO greatly simplifies (and possibly speeds up) the development approach.

Developers will notice the similarities between JDO and Java Serialization. We will look at Serialization which is also a form of transparent persistence and other persistence related technologies in Chapter X.

Transient and Persistent Objects

Java applications instantiate objects using the `new` operator and the constructor. For example:

```
Order myorder= new Order(2828);
myorder.addItem( new LineItem("Core JDO", "$29.99"));
```

When instantiated the virtual machine allocates space on the heap in memory, creates the object and returns a reference. These objects do not represent any persistent data, meaning their state and the state of objects referenced by them is only present in the memory of the virtual machine they are created in. Such objects are called *transient* because they only live as long as the virtual machine process is running, or unless they are garbage collected.

A persistent object is no different from any other object, in that it is created using the same **new** operator and constructor. However a persistent object is an instance of a class that has been marked as *persistence capable* and represents data in some persistent store. The life span of persistent objects is not tied to the life of the virtual machine they are created in.

A persistent object becomes the representation of persistent data as a result of an explicit invocation of JDO API. For example

```
PersistenceManager mgr=pmf.getPersistenceManager();// JDO API
mgr.currentTransaction().begin()/ JDO API
Order myorder= new Order(2828); // Transient object
mgr.makePersistent(myorder); // myorder marked as persistent
mgr.currentTransaction().commit() // JDO API instance myorder
//is written to data store
```

A transient object can also become persistent if it is referenced by as a field in a persistent object. This is known as *persistence by reachability or transitive persistence* (in ODMG terms) and it ensures that when an object is stored in a data store, every item or the graph of items that the object references and collectively represent that objects state are also preserved. This is necessary to recreate the object instance correctly when required. Persistence by reachability is the aspect that results in the appearance of transparent persistence to the developer. The code extract below explains this further:

```
PersistenceManager mgr=pmf.getPersistenceManager();// JDO API
mgr.currentTransaction().begin();// JDO API
Order myorder= new Order(2828); // Transient object
mgr.makePersistent(myorder); // myorder marked as persistent
// other application code here
myorder.addItem( new LineItem("Core JDO", "$29.99"));
mgr.currentTransaction().commit();// JDO API myorder
//including the LineItem is
//also written to data store
```

In the above code segment unlike the **Order** object, the **LineItem** object was never explicitly persisted. However because it is referenced by an object that is persisted, it too will be saved.

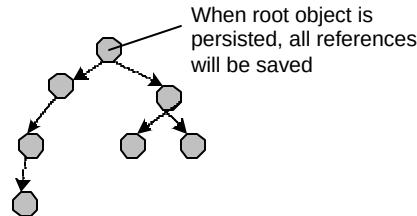


Figure 5: Persistence by reachability

A persistent object can be changed back to a transient object by an explicit call to the JDO API. When a persistent object is marked as transient, any changes made to it will not be reflected in the data store. For example:

```

PersistenceManager mgr=pmf.getPersistenceManager();// JDO API
mgr.currentTransaction().begin(); // JDO API
// other application code here
mgr.makeTransient(myorder) // JDO API
// other application code here

```

A persistent object is also automatically changed to a transient object after it has been persisted. For example, the previous code shown earlier:

```

PersistenceManager mgr=pmf.getPersistenceManager();// JDO API
mgr.currentTransaction().begin(); // JDO API
Order myorder= new Order(2828); // Transient object
mgr.makePersistent(myorder); // myorder marked as persistent
mgr.currentTransaction().commit() // JDO API instance myorder
//is written to data store
myorder.doSomething(); // Transient object

```

Both transient and persistent instances go through different state changes that we will discuss in Chapter X.

Another strategy employed in object persistence is termed as *persistence by inheritance* in which objects instance of a sub class of a persistence capable class can be persisted. JDO is very complete in its support for inheritance. A class can be persistence capable even if its parent is not, a parent can be persistence capable and sub classes may not be and even selective individual classes may be persistence capable in a large inheritance tree. It is up to the requirements specified by the developer.

To summarize the key architectural points surrounding JDO persistence:

- Persistence is orthogonal to the class hierarchy-All user defined objects of any level and of any type may be persisted except those whose state depends on the operating environment (for example system classes like `File`, `Sockets`, `Thread`, `InputStream` etc). This is understandable as objects with such native state depend on or are a reflection off, the state of the operating system they are executing in upon which no guarantees can be placed.
- Persistence is transparent to clients- The logic relating to object persistence is abstracted away. Application code does not move objects to and from stable storage, nor does it deal directly with the underlying data store.
- Persistence is per-object based- Objects of a particular class may be persistent or transient, and in fact there may be in general objects of both kinds for a given class.
- Persistence is based on method invocation- An object is promoted to a persistent state by making use of a `makePersistent()` method.
- Persistence is based on reachability and ensures that all observable properties of a persistent object are preserved even though they may refer to other objects.
- Persistence does not alter object type information and guarantees type safety.

1.4 NON-MANAGED AND MANAGED ENVIRONMENTS

The previous section talked about the need for “*someone*” to call the data store and persistence services. Figure 3 showed this *someone* as being the developer and this needs a bit of elaboration. With the arrival of J2EE, Java applications environments can be broadly grouped into two categories namely non-managed and managed environments.

Non Managed environments

A non-managed environment defines an operational environment for a two-tier application in which an application client directly connects to the resources it is needs. Applications do not depend on any J2EE containers and are typically standalone or based around client-server architectures. In such non managed environments the application developer and the application are responsible for all the interactions with the underlying persistence service. This includes configuring the service, its environment and invoking it at appropriate times in the applications lifetime so that data can be persisted. An application that would use JDO in this manner is shown in Figure 6.

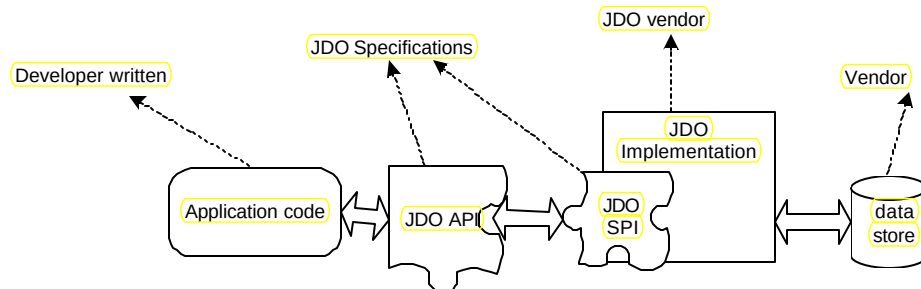


Figure 6: JDO in a non managed environment

Managed environments

A managed environment defines an operational environment for a J2EE-based, multi-tier, web enabled application that accesses Enterprise Information systems (EISs). The application consists of one or more application components E.g. EJBs, JSPs and Servlet that are deployed in *containers*. These containers can be one of the following:

- Web containers that host JSP, Servlets, and static HTML pages
- EJB containers that host EJB components
- Application client containers that host standalone application clients

In a managed environment the J2EE *container* takes the responsibility for configuring the service managing distributed transactions, providing security services and other similar system level functions that would be the responsibility of the application in a non managed environment. As shown in Figure 7, in a managed environment an application component still uses the JDO API, however the JDO implementation now has to support other features so that the application components that are being managed declaratively by the container can work correctly. For example:

- Pooling of the transparent persistence service used JDO from the application components e.g. the JDO `PersistenceManager`
- **Transactions** - The JDO vendor implementation must implement interfaces so that the transactions work as planned and get flushed from the application component to the underlying resources - E.g. the implementation must expose the `javax.transaction.Synchronization` interface.
- If the JDO implementation uses EIS resources it is required to access those stores using resource adaptors in a managed environment. The J2EE Connector Architecture (JCA) defines the standard manner in which J2EE applications can communicate with legacy databases and transactional systems. This is covered in Chapter X.

Typically, and as most JDO vendors use today, a relational database or an object database will be the underlying data store. In this case the JDO implementation is said to be using a *native* resource adaptor.

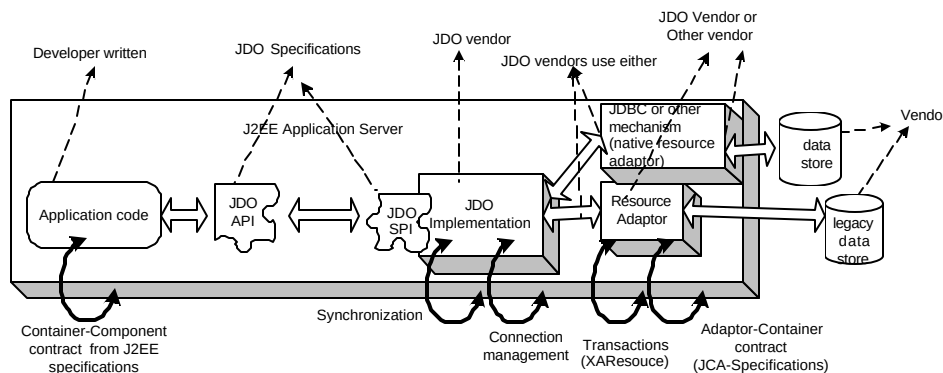


Figure 7: JDO in a managed environment

1.4 ROLES AND RESPONSIBILITIES

From the preceding discussion it is clear that there are different roles (developer, JDO specification, vendor etc) and each has responsibilities that it must fulfill in order for the application to work as designed. Let us look at these in detail

1.4.1 JDO Specifications

The JDO specifications besides being a community standard have five concrete responsibilities:

- ① They define the standard for building object persistence in
- ② The specifications define the standard API for that developers can use in their applications to use transparent persistence
- ③ The specifications also define second API that JDO vendors must implement, called the Service Provider Interface (SPI) that constitutes the JDO environment in a virtual machine.

This is typical design that many Java standards have adopted. For example JDBC defines the API that developers use and another that the JDBC driver provider must implement; JMS defines an API that developer use and another that the vendor providing the messaging server must implement. JNDI and EJB are other examples that

follow the same SPI design.

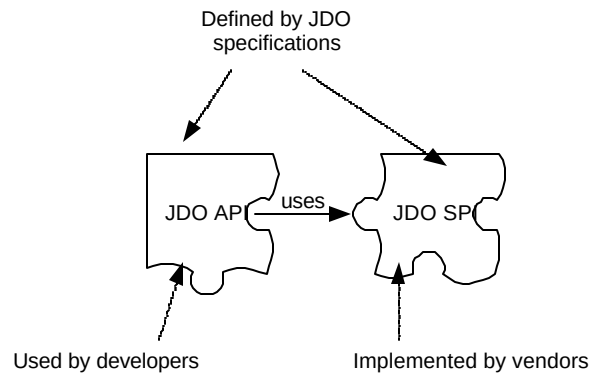


Figure 8 JDO API

- 4 The specifications provide a Reference Implementation of the JDO standard for developers to use and develop applications in.
- 5 The specifications provide a Technology Compatibility Kit (TCK) that can be used by vendors to test compliance with the JDO standards.

1.4.2 Developer

A developer is responsible for:

1. Determining which objects in the object model need to be persisted and writing their Java classes.
2. Marking these classes as *persistence capable* or having the ability to be stored by JDO.

The developer can do this :

- Explicitly by having the class implement an interface defined by the JDO specification- the `javax.jdo.PersistenceCapable`
- Implicitly by using a tool provided by the vendor that takes the compiled Java class code and marks it as being persistence capable. This tool is called a *byte code enhancer* and we will see more about this in Chapter X. The key point to keep in mind is that the developer has the option to *not* implement any special API or persistence related logic in the objects themselves and write the usual Java code. The concept of a byte code enhancer or preprocessor is not new to JDO. As mentioned earlier this was proposed in the 80's and brought to the forefront for Java by the JSPIN research project⁵.

Just because a class is persistence capable does not automatically mean that all instances will be persistent. It only means that the when JDO code is invoked to persist an instance of the class it *will* be persisted to the data store.

3. Using the JDO API as defined in the JDO specifications to persist instances of these classes.

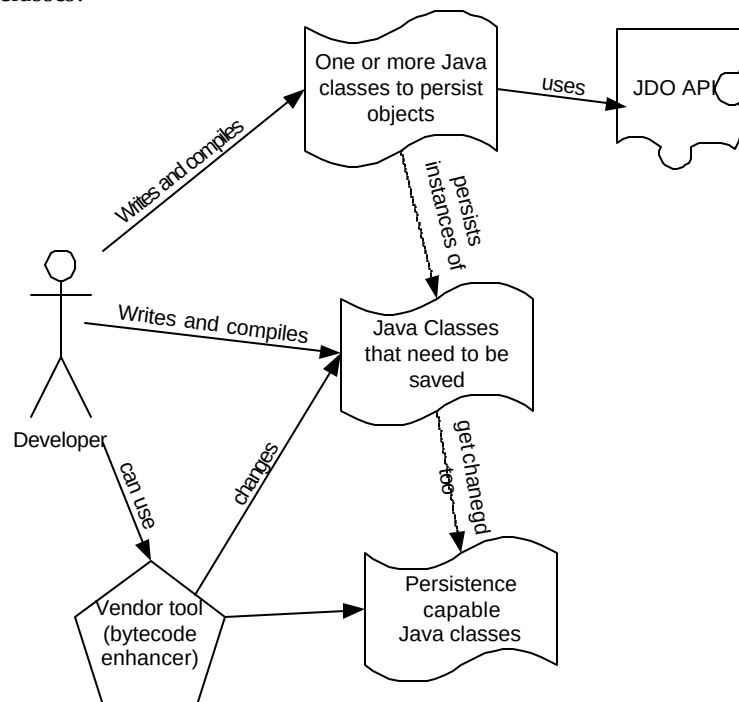


Figure 9 Developer responsibilities

1.4.3 Vendor

The JDO vendor plays an important role by providing the JDO the implementation. The primary responsibility of the vendor is to implement the JDO defined SPI and the JDO specifications. The JDO vendor must also pass the technology compatibility kit (TCK) that is provided by the JDO specifications.

Till now we have used the term vendor as though it referred to a single company. In reality there will be a few different vendors involved.

1. A vendor that provides the JDO implementation. The list of JDO vendors is contained Appendix X

2. A vendor that provides the JCA resource adaptor or JDBC driver to connect to the underlying data store.
3. A vendor that provides the underlying data store. Since JDO vendors are free to use any underlying storage mechanism, the underlying store depends on what the JDO vendor supports and uses. It could be a relational database from another vendor, an object database from the JDO vendor itself, a file database, a hierarchical database, an EIS system etc.
4. If a managed environment is used a J2EE vendor must provide the J2EE application server. For example JDO may be used for bean managed persistence in EJBs, in which case the EJB must be deployed in an EJB container provided by a J2EE server. The relationship between JDO and EJBs is covered in Chapter X

All these logical roles can be fulfilled by a single vendor, in reality there are two categories of JDO vendors:

1. Vendors that provide a JDO implementation using some form of file based persistence or relational database and third party JDBC drivers for that database.
2. Vendors that augment their product offering by providing a JDO implementation. Until recently these implementations have been provided by existing object-database vendors (e.g. Versant), and vendor of object-relational mapping products (e.g. SignSoft). It is possible that in the future, relational database vendors may also augment their offerings by providing JDO implementations along with their servers

It is also typical for JDO vendors to provide implementations that come in two different flavors:

- A stand alone implementation that can be used in a non-managed environment.
- A more expensive version that integrates with J2EE servers from other vendors using JCA technology

SUMMARY

In this chapter we introduced the concepts around which JDO is centered- a standard specification for incorporating persistence facilities into Java applications. We looked at how JDO provides an API that gives developers a Java centric view and insulates the application code from the complexities of the underlying data stores. We also looked at the how JDO has been designed to fit in the non managed and managed environments and the logical roles and responsibilities of the different parties involved.

In the next chapter we will write our first JDO application and introduce you to development with JDO